

Cluster types and runtimes

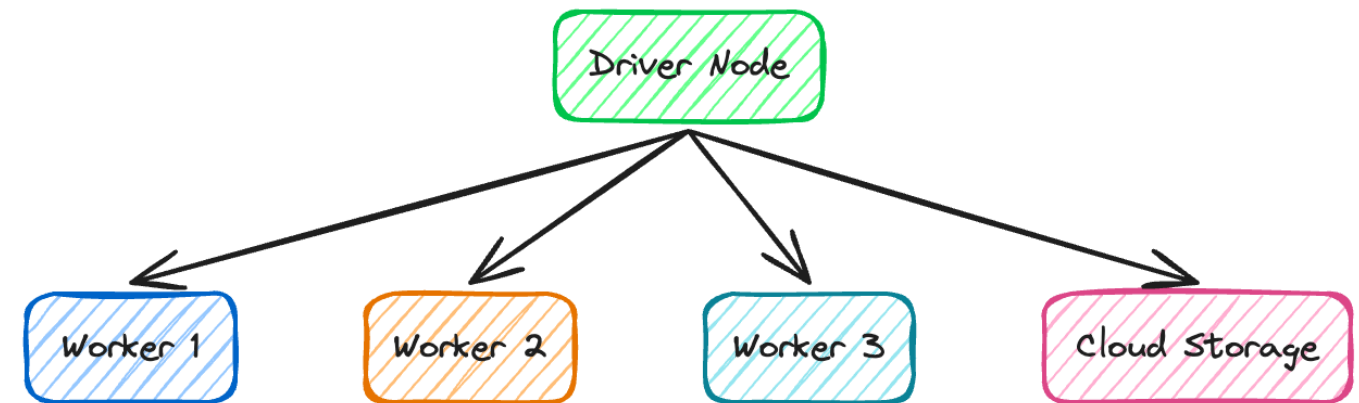
INTRODUCTION TO DATABRICKS LAKEHOUSE



Gang Wang
Senior Data Scientist

What is a cluster?

- A set of virtual machines that run your code
- One **driver** node coordinates the work
- One or more **worker** nodes do the processing
- Clusters read data from cloud storage



All-purpose clusters

- **Interactive** development and ad-hoc queries
- **Shared** across multiple users on a team
- Stay running until you **manually** stop them
- Think of it as your personal car - always available, but you pay even when it's parked

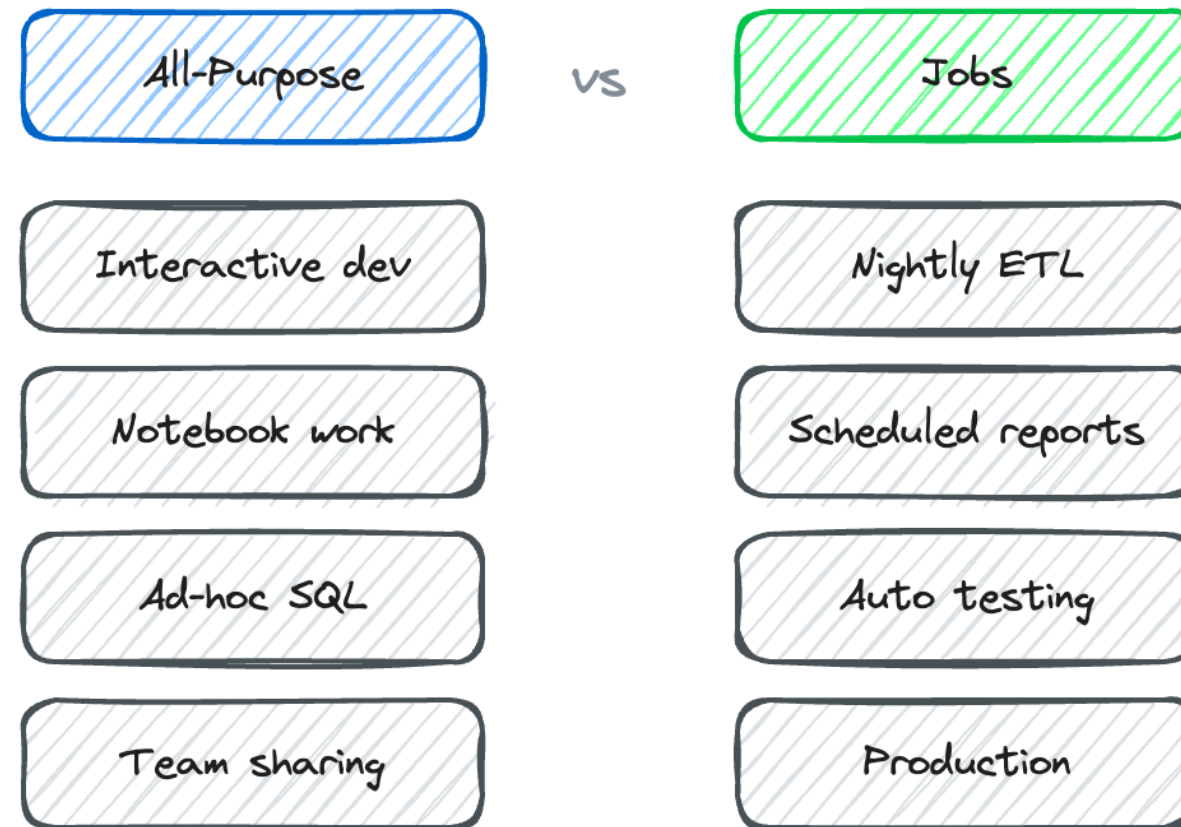


Jobs clusters

- Created **automatically** when a job starts
- Run a **single task**, then terminate
- No manual management required
- **Cost-optimized** for production workloads



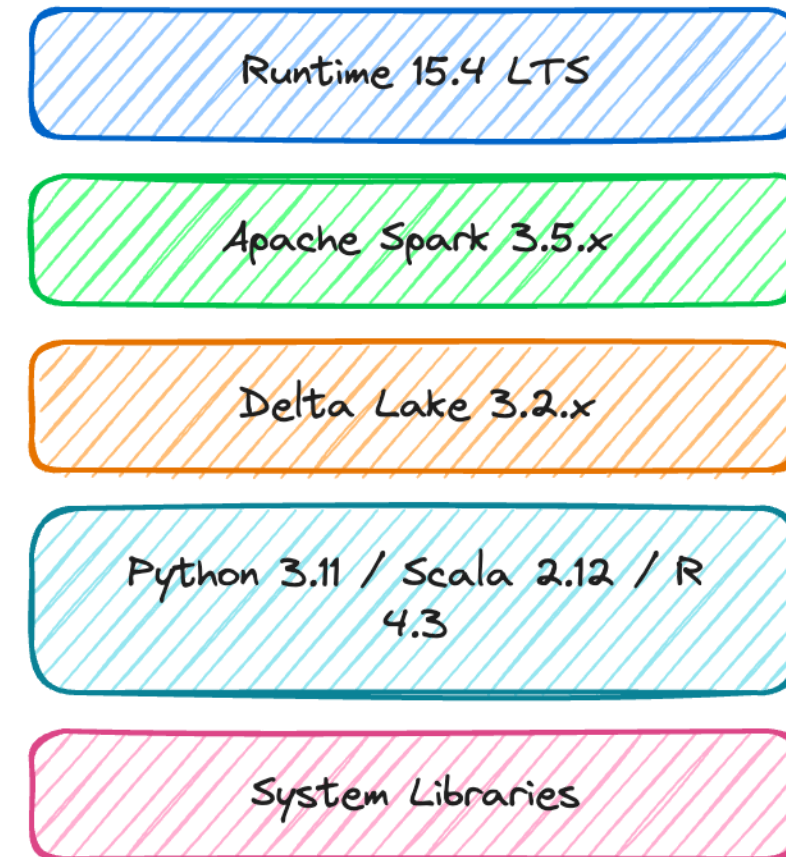
When to use which?



- **Common mistake:** running production pipelines on all-purpose clusters

Databricks Runtime

- **Versioned software stack** that runs on every cluster
- Includes Apache Spark, Delta Lake, Python, R, Scala
- **LTS** versions - long-term support, recommended for production
- **ML Runtime** adds pre-installed ML libraries



Summary

- **All-purpose clusters** - interactive, shared, manually managed
- **Jobs clusters** - automated, single-task, cost-optimized
- **Databricks Runtime** - versioned software stack (use LTS for production)
- Match the cluster type to the workload to control costs

Let's practice!

INTRODUCTION TO DATABRICKS LAKEHOUSE

Managing and configuring clusters

INTRODUCTION TO DATABRICKS LAKEHOUSE



Gang Wang
Senior Data Scientist

Finding your clusters

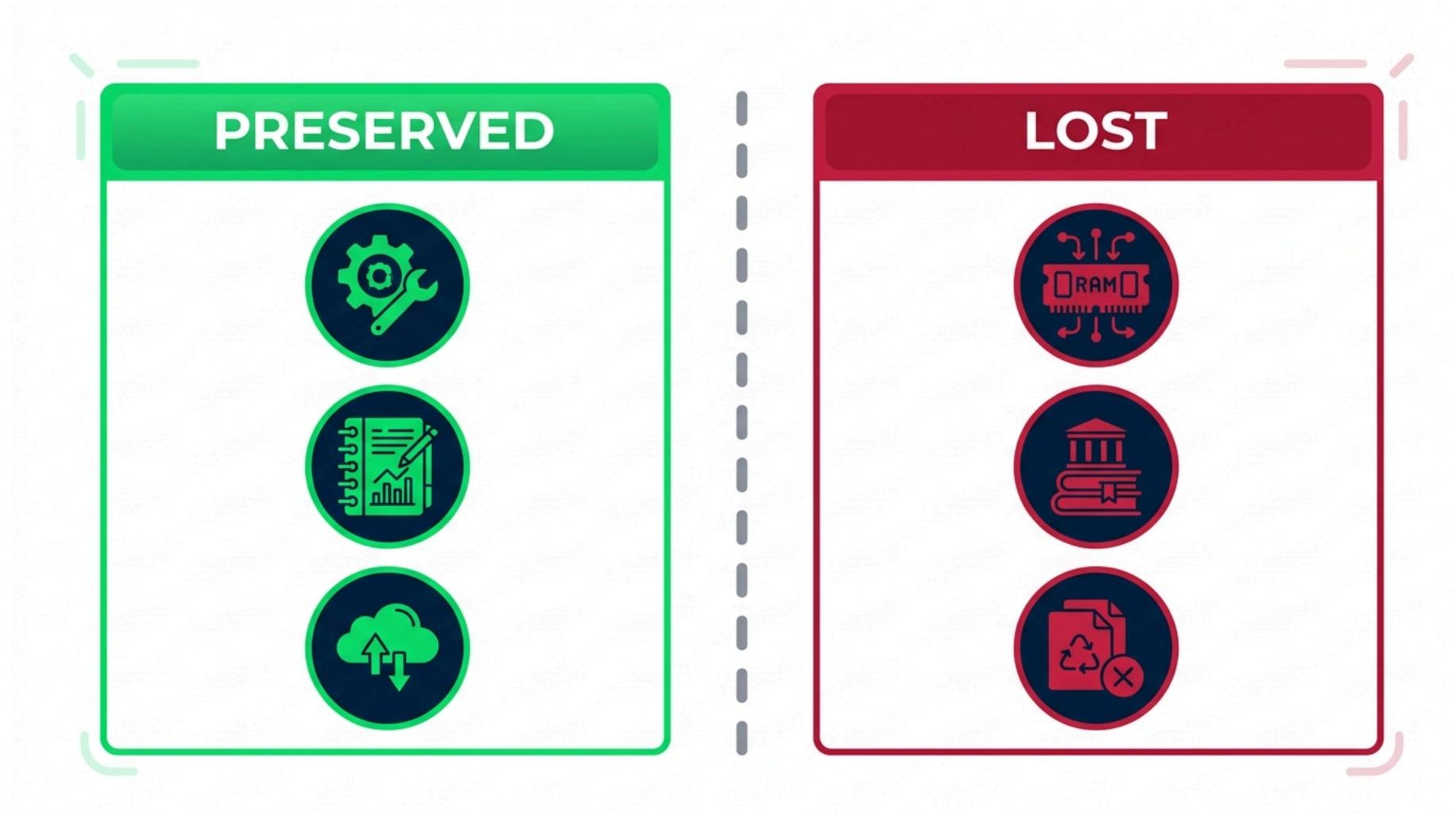
- **All Clusters** - everything in the workspace
- **My Clusters** - ones you created
- **Shared with Me** - clusters others gave you access to
- Filter by state: Running, Terminated



Cluster states



Termination: what you keep and what you lose



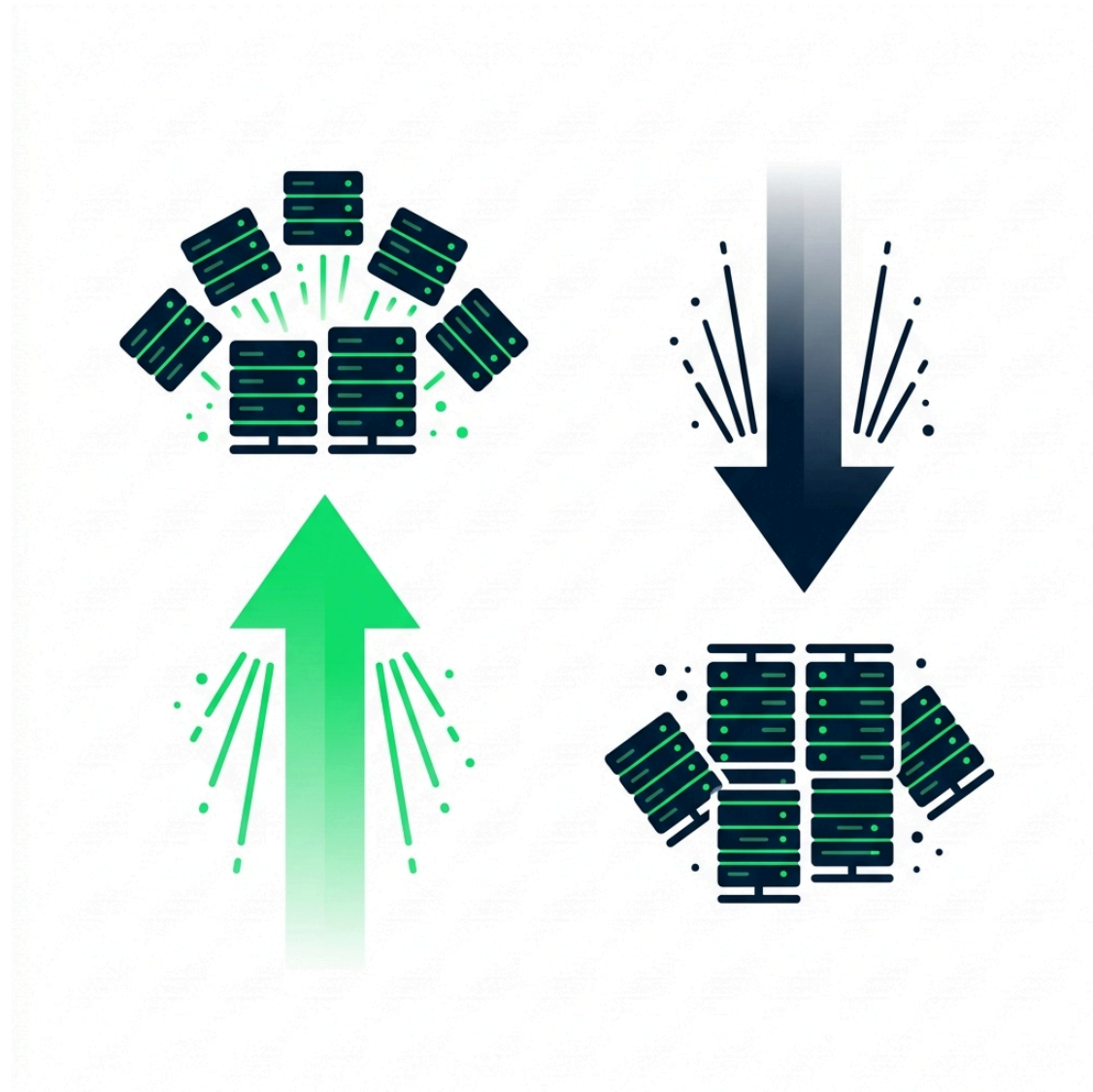
- Termination is **not** deletion - config and notebooks survive
- In-memory data, caches, and pip-installed packages are **cleared**

When to restart

- Installed a new **library** that isn't loading
- Cluster is behaving **unpredictably** or running slowly
- Need to clear **cached data** from memory
- Updated an **init script** or environment variable



Autoscaling and auto-termination



- **Autoscaling** adds or removes workers based on demand
- Handles spiky workloads without over-provisioning
- **Auto-termination** shuts down after a set idle period
- Prevents forgotten clusters from burning budget

Cluster policies

- Admin-defined guardrails for cluster creation
- Limit node types, max workers, and runtime versions
- Enforce auto-termination and tagging
- Users create clusters within the policy bounds

```
{  
  "max_workers": 8,  
  "auto_termination_minutes": 30,  
  "allowed_node_types": [  
    "Standard_DS3_v2",  
    "Standard_DS4_v2"  
  ]  
}
```

Summary

- **Filter** clusters to find what you need quickly
- **Termination** saves costs but clears memory and libraries
- **Restart** to pick up new libraries or troubleshoot issues
- **Autoscaling** and **auto-termination** control costs automatically
- **Cluster policies** let admins enforce standards

Let's practice!

INTRODUCTION TO DATABRICKS LAKEHOUSE

Notebook fundamentals

INTRODUCTION TO DATABRICKS LAKEHOUSE



Gang Wang
Senior Data Scientist

What is a Databricks notebook?



- An interactive document of runnable code cells
- Attached to a **cluster** for execution
- Mix code, results, and documentation in one place
- Supports **Python, SQL, Scala, and R**

Magic commands

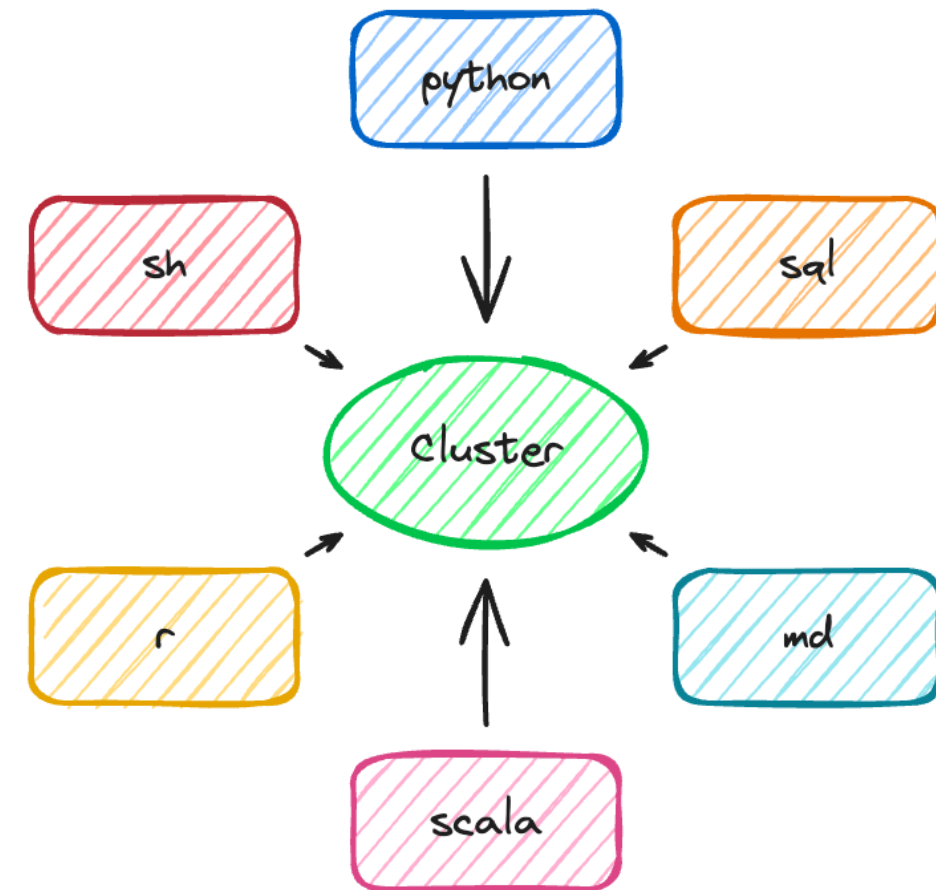
```
# Default language: Python  
df = spark.table("silver_taxi_trips")  
display(df)
```

```
%sql  
  
SELECT COUNT(*) AS total_trips  
FROM silver_taxi_trips
```

```
%md  
  
## Analysis notes  
Revenue is highest in the Northeast region.
```

Available magic commands

Command	Purpose
<code>%python</code>	Run Python code
<code>%sql</code>	Run SQL queries
<code>%scala</code>	Run Scala code
<code>%r</code>	Run R code
<code>%md</code>	Render Markdown
<code>%sh</code>	Run shell commands



Running another notebook with %run

- `%run` executes another notebook in the **same context**
- Functions and variables become available
- Great for **reusable utilities** and shared config

```
# Load shared helper functions
%run /Shared/utils/data_helpers
```

```
# Now use a function defined
# in data_helpers
clean_df = clean_nulls(raw_df)
```

Interpreting results

- Code cells show **output directly below**
- SQL queries render as **interactive tables**
- DataFrames display with built-in **visualization options**
- Errors show **stack traces** with line numbers



Summary

- Notebooks are **interactive documents** attached to clusters
- **Magic commands** let you mix Python, SQL, R, Scala, and Markdown
- `%run` loads functions from other notebooks into your session
- Results render inline with built-in **tables and charts**

Let's practice!

INTRODUCTION TO DATABRICKS LAKEHOUSE

Collaboration and version control

INTRODUCTION TO DATABRICKS LAKEHOUSE



Gang Wang
Senior Data Scientist

Sharing notebooks

- Grant **workspace permissions** to individuals or groups
- Three levels: **Can Read**, **Can Run**, **Can Edit**
- Share via the workspace UI or a direct URL

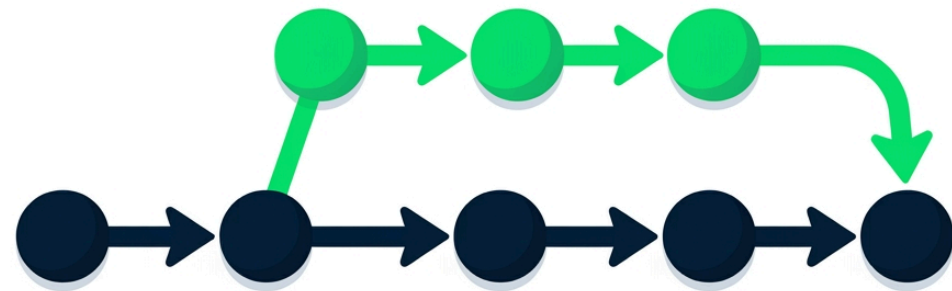
Permission	View	Execute	Modify
Can Read	Yes	No	No
Can Run	Yes	Yes	No
Can Edit	Yes	Yes	Yes

Collaboration features

- **Comments** on specific cells for code review
- **Version history** shows who changed what and when
- But version history is **linear** - no branching or merging

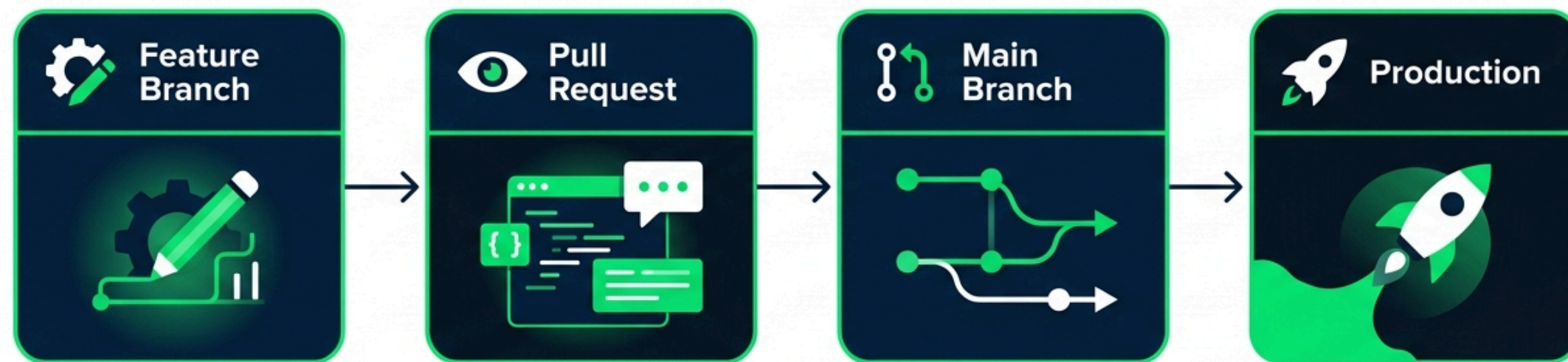


Enter Databricks Repos

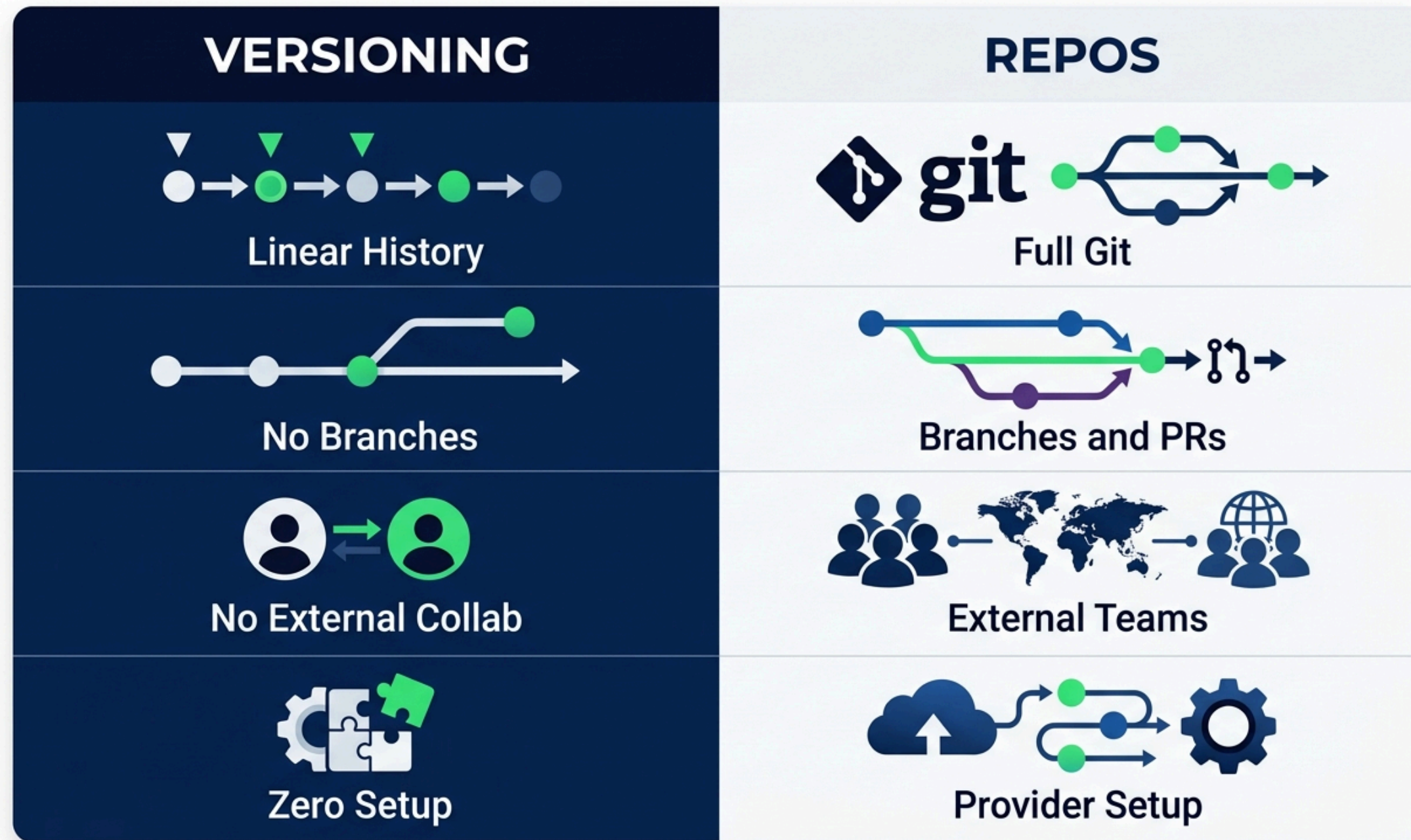


- **Git integration** directly in the workspace
- Connect to GitHub, GitLab, Bitbucket, Azure DevOps
- Clone, commit, push, pull, branch, merge
- Full CI/CD workflows from within Databricks

The CI/CD workflow



Notebook versioning vs. Repos



Summary

- Share notebooks with **Can Read**, **Can Run**, or **Can Edit** permissions
- Built-in **version history** is linear - no branches
- **Databricks Repos** brings full Git integration to the workspace
- Repos enables **CI/CD workflows** with branches, PRs, and automated deployment

Let's practice!

INTRODUCTION TO DATABRICKS LAKEHOUSE

